Programação para Ciência de Dados

Mini Apostila da Disciplina

Curso: MBA em Ciência de Dados
Instituição: Universidade de Fortaleza (UNIFOR)
Professor: Cássio Pinheiro
Carga Horária: 28 horas
Formato: Intensivo
Linguagem: Python
Ambiente: Jupyter Notebook / Google Colab

Apresentação

Esta apostila reúne o conteúdo teórico e prático dos três primeiros módulos da disciplina **Programação para Ciência de Dados**. O material foi elaborado para servir como guia de referência rápida e material de estudo complementar às aulas presenciais.

A disciplina utiliza **Python** como linguagem principal, com foco em bibliotecas amplamente adotadas pela indústria: **NumPy**, **Pandas**, **Matplotlib** e **Seaborn**. O aprendizado é orientado por datasets reais (Titanic, Vendas, COVID-19, IBGE, Filmes), permitindo que os conceitos sejam aplicados em cenários próximos do cotidiano profissional.

Pré-requisitos: Lógica de programação básica. Não é necessária experiência prévia com Python.

Sumário

- Módulo 1 — Fundamentos de Programação com Python
  - 1.1 Tipos de Dados, Operações e Conversões
  - 1.2 Estruturas de Dados Fundamentais
  - 1.3 Estruturas de Controle
  - 1.4 Compreensões e Programação Funcional
  - 1.5 Funções Avançadas
  - 1.6 Tratamento de Erros e Exceções
  - 1.7 Generators e Iteradores
  - 1.8 Manipulação de Arquivos
- Módulo 2 — Manipulação Avançada com Pandas e NumPy
  - 2.1 NumPy — Computação Numérica
  - 2.2 Pandas — Series e DataFrames
  - 2.3 Limpeza e Transformação de Dados
  - 2.4 Agrupamentos e Agregações
  - 2.5 Operações Avançadas
- Módulo 3 — Visualização e Análise Exploratória de Dados
  - 3.1 Matplotlib — Visualizações Fundamentais

    - 3.2 Seaborn — Visualizações Estatísticas
    - 3.3 Visualizações Aplicadas
    - 3.4 Detecção de Outliers
    - 3.5 Resumo da Análise Exploratória
- Glossário
- Referências

Por que Python para Ciência de Dados?

Antes de mergulhar no conteúdo técnico, é importante entender por que Python se tornou a linguagem dominante em Ciência de Dados:

1. **Sintaxe clara** — O código lê quase como pseudocódigo, reduzindo a barreira de entrada.
2. **Ecossistema rico** — Mais de 300.000 pacotes no PyPI, com bibliotecas especializadas para cada etapa do pipeline de dados.
3. **Comunidade ativa** — Documentação extensa, fóruns, tutoriais e cursos abundantes.
4. **Versatilidade** — Da prototipação rápida ao deploy em produção.
5. **Gratuito e open source** — Sem custos de licença.

O Pipeline de Ciência de Dados

Todo projeto de ciência de dados segue, em maior ou menor grau, este fluxo:

```
Definição do Problema → Coleta de Dados → Limpeza → Exploração (EDA) →
Modelagem → Validação → Deploy → Monitoramento
```

Nesta disciplina, os Módulos 1 a 3 cobrem as etapas de **Coleta**, **Limpeza** e **Exploração** — que, juntas, representam cerca de 80% do tempo de um projeto real.

Módulo 1 — Fundamentos de Programação com Python

> **Objetivo de aprendizagem:** Ao final deste módulo, o aluno será capaz de escrever programas em Python utilizando tipos de dados, estruturas de controle, funções, tratamento de erros e boas práticas de programação aplicadas à ciência de dados.

1.1 Tipos de Dados, Operações e Conversões

Todo dado em Python possui um **tipo** que determina quais operações podem ser realizadas sobre ele. Compreender os tipos é o primeiro passo para manipular dados corretamente.

Tipos Primitivos

Tipo	Descrição	Exemplo	Uso em Ciência de Dados
`int`	Números inteiros	`42`, `-7`	Contagens, IDs, classes
`float`	Números decimais	`3.14`, `1e5`	Medidas, preços, médias
`str`	Texto (string)	`"Fortaleza"`	Nomes, categorias, rótulos
`bool`	Booleano	`True`, `False`	Filtros, flags, condições
`NoneType`	Ausência de valor	`None`	Valores ausentes, defaults

Para verificar o tipo de qualquer variável, use `type()`:

```
type(42)              # <class 'int'>
type(3.14)            # <class 'float'>
isinstance(42, int)   # True — verifica se pertence a um tipo
```

Operações Aritméticas

Operador	Operação	Exemplo	Resultado
`+`	Soma	`10 + 3`	`13`
`-`	Subtração	`10 - 3`	`7`
`*`	Multiplicação	`10 * 3`	`30`
`/`	Divisão real	`10 / 3`	`3.333...`
`//`	Divisão inteira	`10 // 3`	`3`
`%`	Módulo (resto)	`10 % 3`	`1`
`**`	Potência	`10 ** 3`	`1000`

> **Dica:** A divisão `/` em Python 3 sempre retorna `float`. Use `//` quando precisar de resultado inteiro (ex: calcular índices).

Conversões de Tipo (Casting)

Em ciência de dados, os dados frequentemente chegam como texto (strings) e precisam ser convertidos para tipos numéricos antes de qualquer análise.

```
int("123")        # String → inteiro: 123
float("3.14")     # String → decimal: 3.14
str(42)           # Inteiro → string: "42"
bool(0)           # → False (0, "", None e [] são "falsy")
bool(1)           # → True  (qualquer valor não-zero é "truthy")
```

> **Atenção:** `int("3.14")` gera erro. Para converter uma string decimal em inteiro, faça `int(float("3.14"))`.

Métodos de String

Strings possuem métodos embutidos que são essenciais na **limpeza de dados** — uma das tarefas mais frequentes em ciência de dados:

```
texto = "  Ciência de Dados - UNIFOR  "

texto.strip()          # Remove espaços nas bordas → "Ciência de Dados - UNIFOR"
texto.lower()          # Tudo minúsculo
texto.upper()          # Tudo maiúsculo
texto.replace('A', 'B')   # Substituir caracteres
texto.split('-')       # Dividir em lista → ["  Ciência de Dados ", " UNIFOR  "]
texto.startswith('C')  # Verifica se começa com 'C'
```

F-strings — a forma moderna e recomendada de formatar texto:

```
nome, nota = "Ana", 9.5
print(f"Aluno(a): {nome}, Nota: {nota:.1f}")
# Saída: Aluno(a): Ana, Nota: 9.5
```

1.2 Estruturas de Dados Fundamentais

Python oferece quatro estruturas de dados nativas. Escolher a estrutura correta para cada situação é uma habilidade fundamental.

Quadro Comparativo

Estrutura	Ordenada	Mutável	Duplicatas	Uso Principal
Lista `[]`	Sim	Sim	Sim	Coleções dinâmicas
Tupla `()`	Sim	Não	Sim	Dados imutáveis
Dicionário `{}`	Sim*	Sim	Chaves: não	Mapeamentos chave-valor
Set `{}`	Não	Sim	Não	Conjuntos, deduplicação

*Dicionários mantêm ordem de inserção a partir do Python 3.7.

Listas

A lista é a estrutura mais versátil do Python. Pense nela como uma **planilha com uma única coluna** — pode guardar qualquer tipo de dado e é fácil de percorrer.

```
numeros = [1, 2, 3, 4, 5]

# Acesso e fatiamento (slicing)
numeros[0]       # Primeiro elemento: 1
numeros[-1]      # Último elemento: 5
numeros[1:4]     # Posições 1 a 3: [2, 3, 4]
numeros[::2]     # De 2 em 2: [1, 3, 5]
numeros[::-1]    # Reverso: [5, 4, 3, 2, 1]
```

Operações comuns:

```
numeros.append(6)         # Adicionar ao final
numeros.extend([7, 8])    # Estender com outra lista
numeros.insert(0, 0)      # Inserir na posição 0
numeros.remove(3)         # Remover primeira ocorrência do valor 3
numeros.pop()             # Remover e retornar o último

sorted(numeros)           # Retorna nova lista ordenada (não altera a
original)
min(numeros), max(numeros), sum(numeros)
```

Desempacotamento — técnica elegante para extrair valores:

```
primeiro, *meio, ultimo = [10, 20, 30, 40, 50]
# primeiro = 10, meio = [20, 30, 40], ultimo = 50
```

List Comprehension — forma concisa e pythonica de criar listas:

```
# Sintaxe: [expressão for item in iterável if condição]

quadrados = [x**2 for x in range(1, 11)]
# [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

pares = [x for x in range(20) if x % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Achatar matriz (lista de listas → lista simples)
matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
achatada = [elem for linha in matriz for elem in linha]
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Dicionários

Dicionários são a estrutura ideal para representar **dados estruturados** — como uma linha de uma tabela, onde cada chave é o nome da coluna.

```
aluno = {
    'nome': 'Ana Silva',
    'idade': 28,
    'curso': 'MBA Ciência de Dados',
    'notas': [8.5, 9.0, 7.5, 8.0]
}

# Acesso seguro (sem risco de KeyError)
aluno.get('email', 'Não informado')

# Atualizar e adicionar
aluno['email'] = 'ana@email.com'
aluno.update({'telefone': '85 99999-0000', 'cidade': 'Fortaleza'})

# Iteração
for chave, valor in aluno.items():
    print(f"  {chave}: {valor}")
```

> **Dica profissional:** Use `.get()` ao invés de `[]` quando não tiver certeza se a chave existe. Isso evita `KeyError` e permite definir um valor padrão.

Dict Comprehension:

```
quadrados = {x: x**2 for x in range(1, 6)}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Inversão de dicionário
invertido = {v: k for k, v in original.items()}
```

Tuplas

Tuplas são como listas, mas **imutáveis** — uma vez criadas, não podem ser alteradas. São ideais para dados que não devem mudar (coordenadas, configurações, retorno de funções).

```
coordenadas = (10.5, 20.3)
x, y = coordenadas    # Unpacking

# Named Tuples — tuplas com campos nomeados (mais legível)
from collections import namedtuple
Ponto = namedtuple('Ponto', ['x', 'y', 'z'])
p = Ponto(1.0, 2.5, 3.7)
print(f"x={p.x}, y={p.y}, z={p.z}")
```

Sets (Conjuntos)

Sets são coleções **sem duplicatas** e **sem ordem**. São perfeitos para operações de conjunto e deduplicação.

```
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

set_a | set_b    # União:              {1, 2, 3, 4, 5, 6, 7, 8}
set_a & set_b    # Interseção:         {4, 5}
set_a - set_b    # Diferença (A - B):  {1, 2, 3}
set_a ^ set_b    # Diferença simétrica: {1, 2, 3, 6, 7, 8}
```

Caso prático — remover duplicatas preservando a ordem:

```
lista = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
unicos = list(dict.fromkeys(lista))
# [3, 1, 4, 5, 9, 2, 6]
```

1.3 Estruturas de Controle

Estruturas de controle determinam o **fluxo de execução** do programa — quais blocos de código serão executados e quantas vezes.

Condicionais

```
if x > 0:
    print("Positivo")
elif x < 0:
    print("Negativo")
else:
    print("Zero")

# Operador ternário — condicional em uma linha
paridade = "par" if x % 2 == 0 else "ímpar"
```

Laços (Loops)

`for` — itera sobre qualquer sequência (lista, string, range, etc.):

```
# Funções auxiliares essenciais:
# enumerate → índice + valor
for i, fruta in enumerate(['maçã', 'banana', 'uva'], start=1):
    print(f"{i}. {fruta}")
```

```
# zip → iterar duas listas em paralelo
for nome, nota in zip(['Ana', 'Bruno'], [9.5, 7.8]):
    print(f"{nome}: {nota}")
```

`while` — repete enquanto a condição for verdadeira:

```
contador = 0
while contador < 5:
    contador += 1
```

Controle Refinado de Laços

```
# break — interrompe o laço imediatamente
for n in range(50, 101):
    if n % 7 == 0:
        print(f"Primeiro divisível por 7: {n}")
        break

# continue — pula para a próxima iteração
for n in range(1, 11):
    if n % 3 == 0:
        continue    # Pula múltiplos de 3
    print(n)

# for/else — o bloco else executa se NÃO houve break
for n in range(50, 55):
    if n % 7 == 0:
        break
else:
    print("Nenhum divisível por 7 foi encontrado")
```

1.4 Compreensões e Programação Funcional

Python oferece um estilo de programação chamado **funcional**, onde funções são tratadas como valores que podem ser passados, retornados e compostos. Esse paradigma é muito utilizado em ciência de dados para transformar e filtrar dados de forma concisa.

Funções Lambda

Funções anônimas (sem nome) de uma única expressão:

```
dobro = lambda x: x * 2
dobro(5)   # 10
```

map, filter e reduce

Função	O que faz	Analogia
`map`	Aplica função a cada elemento	"Transformar tudo"
`filter`	Seleciona elementos por condição	"Filtrar"
`reduce`	Acumula resultado progressivamente	"Resumir em um valor"

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# map — dobrar todos
dobrados = list(map(lambda x: x * 2, numeros))

# filter — apenas pares
pares = list(filter(lambda x: x % 2 == 0, numeros))

# reduce — produto de todos
from functools import reduce
produto = reduce(lambda acc, x: acc * x, numeros)  # 3628800
```

Pipeline funcional — combinar map, filter e reduce:

```
# Soma dos quadrados dos números pares
resultado = reduce(
    lambda acc, x: acc + x,
    map(lambda x: x**2, filter(lambda x: x % 2 == 0, numeros))
)
# (2² + 4² + 6² + 8² + 10²) = 220
```

sorted com key — ordenação customizada:

```
alunos = [('Ana', 9.5), ('Bruno', 7.8), ('Carla', 8.9)]
por_nota = sorted(alunos, key=lambda a: a[1], reverse=True)
# [('Ana', 9.5), ('Carla', 8.9), ('Bruno', 7.8)]
```

1.5 Funções Avançadas

Funções são blocos reutilizáveis de código. Em projetos de ciência de dados, funções bem escritas são essenciais para manter o código **organizado**, **testável** e **reutilizável**.

Funções com Documentação e Type Hints

```python
def calcular_estatisticas(dados: list, incluir_media: bool = True,
                          incluir_mediana: bool = False) -> dict:
    """
    Calcula estatísticas básicas de uma lista de números.

    Args:
        dados: Lista de números.
        incluir_media: Se True, inclui a média no resultado.
        incluir_mediana: Se True, inclui a mediana no resultado.

    Returns:
        Dicionário com as estatísticas calculadas.

    Raises:
        ValueError: Se a lista estiver vazia.
    """
    if not dados:
        raise ValueError("A lista de dados não pode estar vazia")

    resultado = {
        'minimo': min(dados),
        'maximo': max(dados),
        'amplitude': max(dados) - min(dados),
        'quantidade': len(dados)
    }
    if incluir_media:
        resultado['media'] = sum(dados) / len(dados)
    return resultado
```

> **Boas práticas:** Sempre inclua uma **docstring** descrevendo o propósito, parâmetros e retorno da função. Type hints (`dados: list`, `-> dict`) servem como documentação e ajudam IDEs a detectar erros.

*args e **kwargs

Permitem criar funções que aceitam **número variável** de argumentos:

- `*args` — argumentos posicionais extras (recebidos como tupla)
- `**kwargs` — argumentos nomeados extras (recebidos como dicionário)

```python
def relatorio(titulo, *produtos, **detalhes):
    print(f"\n{'='*50}")
    print(f"  {titulo}")
    print(f"{'='*50}")
    for i, prod in enumerate(produtos, 1):
        print(f"  {i}. {prod}")
    for chave, valor in detalhes.items():
        print(f"  {chave}: {valor}")

relatorio("Relatório Mensal",
```

```
            "Notebook", "Monitor", "Teclado",
            vendedor="Carlos", regiao="Nordeste", meta="92%")
```

Decorators

Conceito: Decorators são funções que "envolvem" outras funções, adicionando comportamento extra sem modificar o código original. São usados com a sintaxe `@nome_do_decorator`.

Casos de uso comuns: medir tempo de execução, logging, caching, validação.

```
import time

def medir_tempo(func):
    def wrapper(*args, **kwargs):
        inicio = time.time()
        resultado = func(*args, **kwargs)
        fim = time.time()
        print(f"{func.__name__} executou em {fim - inicio:.4f}s")
        return resultado
    return wrapper

@medir_tempo
def processar_dados(n):
    return sum(i**2 for i in range(n))

processar_dados(1_000_000)
# processar_dados executou em 0.0847s
```

1.6 Tratamento de Erros e Exceções

Em ciência de dados, lidamos constantemente com dados imperfeitos — valores faltantes, tipos incorretos, arquivos corrompidos. O tratamento de erros impede que o programa quebre inesperadamente.

try / except / else / finally

```
def converter_para_numero(valor):
    try:
        resultado = float(valor)       # 1. Tenta executar
    except ValueError:
        print(f"'{valor}' não é um número válido")
        return None                    # 2. Captura erro específico
    except TypeError:
        print(f"Tipo inválido: {type(valor).__name__}")
        return None
    else:
        return resultado               # 3. Executa se NÃO houve erro
    finally:
        pass                           # 4. Executa SEMPRE (limpeza)
```

> **Regra de ouro:** Sempre capture exceções **específicas** (`ValueError`, `TypeError`), nunca use `except:` genérico — isso esconde bugs.

Exceções Customizadas

Criar exceções próprias torna o código mais expressivo e facilita o debugging:

```
class DadosInvalidosError(Exception):
    """Exceção para dados de entrada inválidos."""
    pass

def validar_idade(idade):
    if not isinstance(idade, (int, float)):
        raise DadosInvalidosError(
            f"Idade deve ser numérica, recebeu: {type(idade).__name__}")
    if idade < 0 or idade > 150:
        raise DadosInvalidosError(
            f"Idade fora do intervalo válido (0-150): {idade}")
    return True
```

1.7 Generators e Iteradores

O Problema da Memória

Imagine que você precisa processar 10 milhões de registros. Se carregar tudo na memória como uma lista, o programa pode travar. **Generators** resolvem isso produzindo valores **sob demanda** — um de cada vez.

```
import sys

lista = [x**2 for x in range(100_000)]    # ~800 KB na memória
gen   = (x**2 for x in range(100_000))    # ~200 bytes (!)
```

Generator Functions

Usam `yield` ao invés de `return` — a função "pausa" e retoma de onde parou:

```
def gerar_lotes(dados, tamanho_lote):
    """Divide dados em lotes de tamanho fixo — essencial para big data."""
    for i in range(0, len(dados), tamanho_lote):
        yield dados[i:i + tamanho_lote]

dados = list(range(1, 22))
for i, lote in enumerate(gerar_lotes(dados, 5), 1):
    print(f"Lote {i}: {lote} → soma = {sum(lote)}")
```

1.8 Manipulação de Arquivos

O **context manager** (`with`) garante que o arquivo seja fechado corretamente, mesmo se ocorrer um erro:

```
# Escrita
with open("dados.txt", "w", encoding='utf-8') as f:
    f.write("Linha 1\n")
    f.write("Linha 2\n")

# Leitura completa
with open("dados.txt", "r", encoding='utf-8') as f:
    conteudo = f.read()

# Leitura linha por linha (eficiente para arquivos grandes)
with open("dados.txt", "r", encoding='utf-8') as f:
    for i, linha in enumerate(f, 1):
        print(f"Linha {i}: {linha.strip()}")
```

Modo	Descrição
`'r'`	Leitura (padrão)
`'w'`	Escrita (sobrescreve o arquivo)
`'a'`	Append (adiciona ao final)
`'rb'` / `'wb'`	Binário (leitura/escrita)

Resumo do Módulo 1

Tópico	Conceito-chave
Tipos	`int`, `float`, `str`, `bool`, `None` e conversões com `int()`, `float()`, `str()`
Listas	Coleção ordenada e mutável; list comprehension para criar listas de forma concisa
Dicionários	Pares chave-valor; `.get()` para acesso seguro; dict comprehension
Tuplas/Sets	Tuplas são imutáveis; sets removem duplicatas e fazem operações de conjunto
Controle	`if/elif/else`, `for`, `while`, `break`, `continue`, `enumerate`, `zip`
Funcional	`lambda`, `map`, `filter`, `reduce`, pipelines funcionais
Funções	`*args`, `**kwargs`, decorators, type hints, docstrings
Erros	`try/except/else/finally`, exceções customizadas
Generators	`yield`, processamento sob demanda, economia de memória
Arquivos	`with open()`, modos `r/w/a`, encoding UTF-8

Módulo 2 — Manipulação Avançada com Pandas e NumPy

> **Objetivo de aprendizagem:** Ao final deste módulo, o aluno será capaz de criar e manipular arrays NumPy, carregar e limpar datasets com Pandas, realizar agrupamentos, joins e transformações complexas em dados tabulares.

2.1 NumPy — Computação Numérica

Por que NumPy?

O NumPy (Numerical Python) é a base de todo o ecossistema de ciência de dados em Python. Ele fornece o **ndarray** — um array multidimensional homogêneo que é 10 a 100 vezes mais rápido que listas Python para operações matemáticas.

Característica	NumPy Array	Lista Python
Velocidade	10-100x mais rápido	Lento (loops)
Memória	Eficiente (contíguo)	Maior overhead
Tipo de dados	Homogêneo	Heterogêneo
Operações	Vetorizadas	Requer loop explícito

Criação de Arrays

```
import numpy as np

# A partir de listas
arr1d = np.array([1, 2, 3, 4, 5])
arr2d = np.array([[1, 2, 3], [4, 5, 6]])

# Arrays especiais
np.zeros((3, 4))          # Matriz 3×4 de zeros
np.ones((2, 3))           # Matriz 2×3 de uns
np.eye(3)                 # Matriz identidade 3×3
np.full((3, 3), 7)        # Preenchida com valor 7

# Sequências
np.arange(0, 10, 2)       # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)      # 5 valores igualmente espaçados entre 0 e 1

# Aleatórios (com reprodutibilidade)
np.random.seed(42)
np.random.rand(3, 4)      # Distribuição uniforme [0, 1)
np.random.randn(3, 4)     # Distribuição normal padrão
np.random.randint(0, 10, size=(3, 4))  # Inteiros aleatórios
```

> **Dica:** Sempre use `np.random.seed()` para garantir **reprodutibilidade** dos resultados — essencial em trabalhos científicos.

Operações Vetorizadas

A grande vantagem do NumPy: operações são aplicadas **elemento a elemento** sem necessidade de loops.

```
a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])

a + b       # [11, 22, 33, 44, 55]
a * b       # [10, 40, 90, 160, 250]
a ** 2      # [1, 4, 9, 16, 25]
a * 2       # [2, 4, 6, 8, 10]  — operação com escalar
a > 3       # [False, False, False, True, True]  — comparação vetorizada
```

Broadcasting

Conceito: Broadcasting permite operações entre arrays de **dimensões diferentes**, estendendo automaticamente o array menor.

```
matriz = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
vetor = np.array([10, 20, 30])

# Cada linha da matriz soma com o vetor:
matriz + vetor
# [[11, 22, 33],
#  [14, 25, 36],
#  [17, 28, 39]]
```

Aplicação prática — normalização z-score (muito usada em Machine Learning):

```
media = matriz.mean(axis=0)    # Média por coluna
std = matriz.std(axis=0)       # Desvio padrão por coluna
normalizado = (matriz - media) / std  # Broadcasting automático
```

Indexação e Fatiamento

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Indexação booleana — filtrar por condição
arr[arr > 5]              # [6, 7, 8, 9, 10]
arr[arr % 2 == 0]         # [2, 4, 6, 8, 10]
arr[(arr > 3) & (arr < 8)]  # [4, 5, 6, 7]
```

```
# Fancy indexing — selecionar por posições específicas
arr[[0, 2, 4]]            # [1, 3, 5]
```

Operações Estatísticas

```
np.mean(arr)                        # Média
np.median(arr)                      # Mediana
np.std(arr)                         # Desvio padrão
np.var(arr)                         # Variância
np.percentile(arr, [25, 50, 75])    # Quartis
```

Conceito de eixo (axis) — fundamental para operações em matrizes:

Parâmetro	Significado	Resultado
`axis=None`	Todo o array	Um único valor
`axis=0`	Ao longo das linhas	Um valor **por coluna**
`axis=1`	Ao longo das colunas	Um valor **por linha**

Reshape, Stack e Concatenação

```
arr = np.arange(1, 13)
arr.reshape(3, 4)            # Reformatar para 3 linhas × 4 colunas
arr.reshape(4, 3).T          # Transposta

np.concatenate([a, b])       # Concatenar arrays
np.vstack([a, b])            # Empilhar verticalmente
np.hstack([a, b])            # Empilhar horizontalmente

# np.where — IF vetorizado (muito útil!)
notas = np.array([85, 42, 73, 91, 56])
np.where(notas >= 70, 'Aprovado', 'Reprovado')
# ['Aprovado', 'Reprovado', 'Aprovado', 'Aprovado', 'Reprovado']
```

Álgebra Linear

```
A = np.array([[1, 2], [3, 4]])

np.dot(A, B)                # Produto de matrizes (ou A @ B)
np.linalg.det(A)             # Determinante
np.linalg.inv(A)             # Inversa
np.linalg.eig(A)             # Autovalores e autovetores
np.linalg.solve(A, b)        # Resolver sistema linear Ax = b
```

2.2 Pandas — Series e DataFrames

A Ferramenta Central

Se NumPy é o motor, **Pandas** é o painel de controle. É a biblioteca mais importante para manipulação de dados tabulares em Python. Dois objetos fundamentais:

- **Series** — array 1D rotulado (uma coluna)
- **DataFrame** — tabela 2D rotulada (planilha inteira)

Series

```
import pandas as pd

notas = pd.Series(
    [9.5, 7.8, 8.2, 6.5, 9.0],
    index=['Ana', 'Bruno', 'Carla', 'Diego', 'Eva'],
    name='Notas'
)

notas.mean()          # Média: 8.2
notas.median()        # Mediana: 8.2
notas[notas >= 7]     # Filtragem booleana: Ana, Bruno, Carla, Eva
notas * 10            # Operação vetorizada: notas de 0 a 100

# Categorizar com pd.cut
pd.cut(notas, bins=[0, 5, 7, 9, 10],
       labels=['Insuficiente', 'Regular', 'Bom', 'Excelente'])
```

DataFrame — Criação e Inspeção

```
# De dicionário
df = pd.DataFrame({
    'nome': ['Ana', 'Bruno', 'Carlos'],
    'idade': [25, 30, 35],
    'cidade': ['Fortaleza', 'Recife', 'Salvador']
})

# De arquivo (o mais comum na prática)
df = pd.read_csv('dados.csv')
```

Inspeção — os primeiros comandos em qualquer análise:

```
df.head()                     # Primeiras 5 linhas
df.shape                      # (linhas, colunas)
df.info()                     # Tipos, nulos, memória
df.describe()                 # Estatísticas descritivas (numéricas)
```

```
df.describe(include='object') # Estatísticas de colunas categóricas
df.dtypes                     # Tipo de cada coluna
df.columns                    # Nomes das colunas
```

Seleção de Dados

```
# Colunas
df['idade']                      # Uma coluna → Series
df[['nome', 'idade']]           # Múltiplas colunas → DataFrame

# Linhas por posição
df.iloc[0:3]                     # Linhas 0, 1, 2

# Linhas por condição (filtragem)
df[df['idade'] > 25]
df[(df['idade'] > 20) & (df['cidade'] == 'Fortaleza')]

# Combinado: condição + colunas
df.loc[df['idade'] > 25, ['nome', 'cidade']]
```

> **Regra prática:** Use `iloc` para posição numérica, `loc` para labels e condições.

2.3 Limpeza e Transformação de Dados

A limpeza de dados consome até **80% do tempo** de um projeto de ciência de dados. Dados do mundo real são imperfeitos — possuem valores ausentes, duplicatas, tipos incorretos e inconsistências.

Diagnóstico

```
# Valores nulos
df.isnull().sum()                                    # Contagem por
coluna
(df.isnull().sum() / len(df) * 100).round(2)         # Percentual

# Duplicatas
df.duplicated().sum()

# Valores únicos
for col in df.columns:
    print(f"{col}: {df[col].nunique()} valores únicos")
```

Tratamento de Valores Ausentes

```
# Preencher com mediana (mais robusto que média para dados com outliers)
df['Age'] = df['Age'].fillna(df['Age'].median())
```

```
# Remover linhas com nulos
df.dropna()                     # Qualquer nulo
df.dropna(subset=['idade'])     # Apenas se coluna específica for nula
```

> **Quando usar mediana vs média?** A **mediana** é preferível quando há outliers, pois não é influenciada por valores extremos. A **média** é adequada para distribuições simétricas sem outliers.

Feature Engineering

Criar novas variáveis a partir das existentes é uma das habilidades mais valiosas em ciência de dados:

```
# pd.cut — faixas fixas definidas pelo analista
df['FaixaEtaria'] = pd.cut(df['Age'],
    bins=[0, 12, 18, 35, 60, 100],
    labels=['Criança', 'Adolescente', 'Jovem', 'Adulto', 'Idoso'])

# pd.qcut — faixas com quantidade igual de observações (quartis)
df['TarifaCategoria'] = pd.qcut(df['Fare'], q=4,
    labels=['Econômica', 'Básica', 'Intermediária', 'Premium'])

# map — substituir valores
df['Sex'] = df['Sex'].map({'male': 'Masculino', 'female': 'Feminino'})

# apply — aplicar função customizada
def classificar(receita):
    if receita < 100000: return 'Baixa'
    elif receita < 300000: return 'Média'
    return 'Alta'

df['ClasseReceita'] = df['Receita'].apply(classificar)
```

Operações com Strings

```
df['col'].str.upper()              # MAIÚSCULAS
df['col'].str.lower()              # minúsculas
df['col'].str.strip()              # Remover espaços
df['col'].str.contains('texto')    # Buscar padrão
df['col'].str.replace('antigo', 'novo')
df['col'].str.split(' ', expand=True)  # Dividir em colunas
```

2.4 Agrupamentos e Agregações

GroupBy — Split-Apply-Combine

O `groupby` é uma das operações mais poderosas do Pandas. Ele divide os dados em grupos, aplica uma função a cada grupo e combina os resultados.

Analogia: É como criar uma tabela dinâmica no Excel, mas com muito mais flexibilidade.

```
# Agregação simples
df.groupby('Pclass')['Survived'].mean()

# Múltiplas estatísticas
df.groupby('Pclass')['Survived'].agg(['mean', 'sum', 'count'])

# Múltiplas colunas de agrupamento
df.groupby(['Pclass', 'Sex'])['Survived'].agg(['mean', 'count'])

# Named Aggregation (sintaxe moderna do Pandas)
df.groupby('Pclass').agg(
    idade_media=('Age', 'mean'),
    idade_mediana=('Age', 'median'),
    tarifa_media=('Fare', 'mean'),
    total=('PassengerId', 'count')
)
```

2.5 Operações Avançadas

Pivot Tables

Analogia: Equivalente às tabelas dinâmicas do Excel.

```
pivot = pd.pivot_table(
    df_vendas,
    values='Receita',        # O que agregar
    index='Regiao',          # Linhas
    columns='Produto',       # Colunas
    aggfunc='sum',           # Função de agregação
    fill_value=0             # Preencher vazios com 0
)
```

Merge / Join

Conceito: Combinar DataFrames baseado em colunas comuns — equivalente ao `JOIN` do SQL.

Tipo	SQL equivalente	Descrição
`inner`	`INNER JOIN`	Apenas registros com match em ambos
`left`	`LEFT JOIN`	Todos da esquerda + matches da direita
`right`	`RIGHT JOIN`	Todos da direita + matches da esquerda

Tipo	SQL equivalente	Descrição
`outer`	`FULL OUTER JOIN`	Todos de ambos (união)

```
# Inner join (padrão)
pd.merge(df_clientes, df_pedidos, on='cliente_id')

# Left join
pd.merge(df_clientes, df_pedidos, on='cliente_id', how='left')

# Colunas com nomes diferentes
pd.merge(df1, df2, left_on='id', right_on='pessoa_id')
```

Window Functions

Conceito: Funções que calculam valores baseados em uma "janela" de observações (registros vizinhos ou acumulados). Essenciais para análise de séries temporais.

```
# Média móvel de 3 períodos — suaviza flutuações
df['MediaMovel_3m'] = df['Receita'].rolling(window=3).mean()

# Média acumulada — como a média evolui ao longo do tempo
df['MediaAcumulada'] = df['Receita'].expanding().mean()

# Variação percentual período a período
df['Variacao_%'] = df['Receita'].pct_change() * 100

# Lag (valor do período anterior) — comparar com mês passado
df['Receita_Anterior'] = df['Receita'].shift(1)
```

Method Chaining

Técnica para encadear operações de forma limpa, criando **pipelines de transformação**:

```
resultado = (
    df_vendas
    .query("Receita > 50000")
    .groupby('Regiao')
    .agg(
        receita_total=('Receita', 'sum'),
        lucro_total=('Lucro', 'sum'),
        num_vendas=('Receita', 'count')
    )
    .assign(margem_pct=lambda x: x['lucro_total'] / x['receita_total'] *
100)
    .sort_values('receita_total', ascending=False)
```

```
    .round(2)
)
```

Resumo do Módulo 2

Tópico	Conceito-chave
NumPy Arrays	`np.array`, `np.zeros`, `np.ones`, `np.arange`, `np.linspace`
Vetorização	Operações sem loop: `a + b`, `a * 2`, `a > 3`
Broadcasting	Operações entre arrays de dimensões diferentes
Indexação	Booleana (`arr[arr > 5]`), fancy (`arr[[0,2,4]]`)
Series	Array 1D rotulado, filtragem, `pd.cut`
DataFrame	Tabela 2D: `head()`, `info()`, `describe()`, `iloc`/`loc`
Limpeza	`fillna()`, `dropna()`, `duplicated()`, feature engineering
GroupBy	Split-Apply-Combine, `agg()`, named aggregation
Pivot	`pd.pivot_table()` — tabela dinâmica
Merge	Inner, left, right, outer — combinar DataFrames
Window	`rolling()`, `expanding()`, `shift()`, `pct_change()`
Chaining	Encadear operações: `.query().groupby().agg().sort_values()`

Módulo 3 — Visualização e Análise Exploratória de Dados

> **Objetivo de aprendizagem:** Ao final deste módulo, o aluno será capaz de criar visualizações informativas com Matplotlib e Seaborn, realizar análise exploratória completa, detectar outliers e produzir dashboards analíticos.

Princípios de Visualização

A visualização transforma números em **insights**. Um bom gráfico comunica em segundos o que uma tabela com mil linhas não consegue.

Os 4 princípios fundamentais:

1. **Clareza** — A mensagem principal deve ser imediatamente aparente
2. **Precisão** — Representar dados fielmente, sem distorcer escalas
3. **Eficiência** — Escolher o tipo certo de gráfico para cada dado
4. **Estética** — Cores harmoniosas, tipografia legível, espaçamento adequado

Guia Rápido — Qual Gráfico Usar?

Objetivo	Gráfico Recomendado
Distribuição de uma variável	Histograma, KDE, Box Plot
Comparar categorias	Barras (vertical ou horizontal)
Evolução no tempo	Gráfico de linha
Relação entre 2 variáveis	Scatter plot
Composição (partes de um todo)	Barras empilhadas, pizza (com moderação)
Correlação entre variáveis	Heatmap
Visão geral multivariada	Pair plot, FacetGrid
Detectar outliers	Box plot

3.1 Matplotlib — Visualizações Fundamentais

Matplotlib é a biblioteca **base** de visualização em Python. Todas as outras (Seaborn, Plotly, etc.) são construídas sobre ela.

Configuração recomendada:

```
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 11
```

Histograma

Quando usar: Entender a distribuição de uma variável contínua.

```
plt.figure(figsize=(10, 6))
plt.hist(df['Age'], bins=30, alpha=0.7, color='skyblue',
edgecolor='black')
plt.axvline(df['Age'].mean(), color='red', linestyle='--', label='Média')
plt.axvline(df['Age'].median(), color='green', linestyle='--',
label='Mediana')
plt.xlabel('Idade')
plt.ylabel('Frequência')
plt.title('Distribuição de Idades')
plt.legend()
plt.show()
```

Histogramas sobrepostos — para comparar distribuições entre grupos:

```
for sobreviveu, label, cor in [(1, 'Sobreviveu', 'steelblue'), (0, 'Não',
'salmon')]:
    subset = df[df['Survived'] == sobreviveu]['Age']
    plt.hist(subset, bins=25, alpha=0.6, label=label, color=cor,
edgecolor='black')
plt.legend()
```

> **Como escolher o número de bins?** Regra prática: entre 10 e 30 bins geralmente funciona. Poucos demais escondem detalhes; muitos demais criam ruído.

Gráfico de Barras

Quando usar: Comparar valores entre categorias.

Variações: vertical, horizontal (nomes longos), agrupado (comparar séries), empilhado (composição).

```
# Barras com valores anotados
bars = plt.bar(categorias, valores, color=['gold', 'silver', '#CD7F32'])
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.1%}', ha='center', va='bottom', fontweight='bold')
```

Subplots — Múltiplos Gráficos

Quando usar: Criar dashboards compactos ou comparações lado a lado.

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

axes[0, 0].hist(df['Age'], bins=30)
axes[0, 0].set_title('Distribuição de Idades')

axes[0, 1].hist(df['Fare'], bins=30)
axes[0, 1].set_title('Distribuição de Tarifas')

plt.tight_layout()   # Ajusta espaçamento automaticamente
plt.show()
```

Gráfico de Linha

Quando usar: Séries temporais, tendências ao longo do tempo.

```
plt.plot(datas, receita, 'o-', alpha=0.4, label='Receita mensal')
plt.plot(datas, media_movel, '-', linewidth=2.5, label='Média móvel 3m')
```

```
plt.fill_between(datas, receita, alpha=0.2)
```

3.2 Seaborn — Visualizações Estatísticas

Seaborn é construída sobre Matplotlib e oferece uma API de **alto nível** para gráficos estatísticos com menos código e melhor estética por padrão.

Box Plot

Conceito: Mostra a distribuição resumida em 5 números: mínimo, Q1, mediana, Q3, máximo, além de outliers.

Como interpretar:

- **Caixa** = IQR (Q1 a Q3) → onde estão 50% dos dados
- **Linha central** = Mediana
- **Whiskers** = 1.5 × IQR
- **Pontos isolados** = Outliers

```
sns.boxplot(x='Pclass', y='Age', hue='Survived', data=df, palette='Set2')
```

Violin Plot

Combina box plot com **KDE** — mostra a forma completa da distribuição, incluindo multimodalidade (múltiplos picos).

```
sns.violinplot(x='Pclass', y='Fare', hue='Sex', data=df, split=True)
```

Heatmap

Quando usar: Visualizar matrizes (correlação, tabelas cruzadas).

```
correlacao = df[['Pclass', 'Age', 'Fare', 'Survived']].corr()
sns.heatmap(correlacao, annot=True, cmap='coolwarm', center=0)
```

Escalas de cores:

- **Sequencial** (Blues, YlOrRd) → dados ordenados
- **Divergente** (coolwarm, RdBu) → ponto central importante (ex: correlação)
- **Qualitativa** (Set1, Set2) → categorias distintas

Pair Plot

Conceito: Gera automaticamente scatter plots e distribuições para **todas as combinações** de variáveis numéricas. Ideal para a fase inicial da EDA.

```
sns.pairplot(
    df[['Age', 'Fare', 'Pclass', 'Survived']],
    hue='Survived',
    palette={0: '#e74c3c', 1: '#2ecc71'},
    diag_kind='kde'
)
```

KDE (Kernel Density Estimation)

Conceito: Versão suavizada do histograma — estima a curva de densidade de probabilidade. Ideal para **comparar distribuições** entre grupos.

```
for sobreviveu, label, cor in [(0, 'Não', '#e74c3c'), (1, 'Sim',
'#2ecc71')]:
    subset = df[df['Survived'] == sobreviveu]['Age']
    sns.kdeplot(subset, label=label, color=cor, fill=True, alpha=0.3)
```

Scatter com Regressão (lmplot)

Combina scatter plot com **linha de regressão** — permite visualizar tendência e força da relação linear.

```
sns.lmplot(data=df, x='Age', y='Fare', hue='Survived',
           scatter_kws={'alpha': 0.4}, line_kws={'linewidth': 2})
```

FacetGrid

Conceito: Divide um gráfico em **subpainéis** por categorias. Útil para visualizar como um padrão varia entre subgrupos.

```
g = sns.FacetGrid(df, col='Pclass', row='Sex', hue='Survived',
                  height=3.5, aspect=1.3)
g.map(plt.hist, 'Age', alpha=0.6, bins=20, edgecolor='black')
g.add_legend()
```

3.3 Visualizações Aplicadas

Evolução Temporal com Média Móvel

Combinar os dados brutos com uma média móvel ajuda a **separar tendência de ruído**:

```
ax.plot(datas, receita, 'o-', alpha=0.4, label='Receita mensal')
ax.plot(datas, mm3_receita, '-', linewidth=2.5, label='Média móvel 3m')
```

Barras Empilhadas Normalizadas

Mostram a **composição percentual** de cada categoria:

```
pivot_norm = pivot.div(pivot.sum(axis=1), axis=0) * 100
pivot_norm.plot(kind='barh', stacked=True)
```

Dashboards com Subplots

Combinar 4 ou mais gráficos em um único painel fornece uma **visão panorâmica** dos dados:

```
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
# Preencher cada axes[i, j] com um gráfico diferente
plt.suptitle('Dashboard Analítico', fontsize=16, fontweight='bold')
plt.tight_layout()
```

3.4 Detecção de Outliers

O que são Outliers?

Outliers são valores que se **desviam significativamente** do padrão dos demais dados. Podem ser erros de medição, erros de entrada ou eventos raros legítimos.

Método IQR (Interquartile Range)

O método mais utilizado para detecção de outliers:

```
Q1 = percentil 25%
Q3 = percentil 75%
IQR = Q3 - Q1

Limite inferior = Q1 - 1.5 × IQR
Limite superior = Q3 + 1.5 × IQR

Outlier = qualquer valor fora desses limites
```

```
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
IQR = Q3 - Q1
```

```
limite_inf = Q1 - 1.5 * IQR
limite_sup = Q3 + 1.5 * IQR
outliers = df[(df['Fare'] < limite_inf) | (df['Fare'] > limite_sup)]
```

Decisão: Remover ou Manter?

Situação	Decisão
Erro de digitação / entrada	**Remover** ou corrigir
Erro de medição	**Remover**
Valor impossível (idade = -5)	**Remover**
Evento raro mas legítimo	**Manter**
Amostra pequena	**Manter** (poucos dados para descartar)
Interesse específico em extremos	**Manter**

3.5 Resumo da Análise Exploratória

Função Automatizada de EDA

Uma boa prática é criar uma função reutilizável que faça o resumo inicial de qualquer dataset:

```
def resumo_eda(df, nome="Dataset"):
    print(f"{'='*60}")
    print(f"  RESUMO EDA — {nome}")
    print(f"{'='*60}")
    print(f"Dimensões: {df.shape[0]} linhas × {df.shape[1]} colunas")
    print(f"Memória: {df.memory_usage(deep=True).sum() / 1024:.1f} KB")
    print(f"Nulos: {df.isnull().sum().sum()}")
    print(f"Duplicatas: {df.duplicated().sum()}")

    for col in df.select_dtypes(include='number').columns:
        skew = df[col].skew()
        simetria = "simétrica" if abs(skew) < 0.5 else (
            "assimetria positiva" if skew > 0 else "assimetria negativa")
        print(f"  {col}: média={df[col].mean():.2f}, "
              f"std={df[col].std():.2f}, skew={skew:.2f} → {simetria}")
```

Checklist EDA — Os 8 Passos

#	Etapa	Ferramenta
1	Dimensões e tipos	`df.shape`, `df.dtypes`, `df.info()`
2	Valores ausentes	`df.isnull().sum()`, percentual por coluna
3	Duplicatas	`df.duplicated().sum()`

#	Etapa	Ferramenta
4	Estatísticas descritivas	`df.describe()`, assimetria (skewness)
5	Distribuições	Histogramas, KDE, box plots
6	Relações entre variáveis	Scatter plots, correlação, pair plots
7	Segmentações	GroupBy, comparações entre categorias
8	Outliers	IQR, box plots, limites inferior/superior

Resumo do Módulo 3

Tópico	Conceito-chave
Matplotlib	`plt.hist`, `plt.bar`, `plt.plot`, `plt.subplots`, `plt.pie`
Seaborn	`sns.boxplot`, `sns.violinplot`, `sns.heatmap`, `sns.countplot`
Pair Plot	Visão geral de relações: `sns.pairplot(hue=...)`
KDE	Distribuição suavizada: `sns.kdeplot(fill=True)`
Implot	Scatter + regressão: `sns.lmplot(x, y, hue=...)`
FacetGrid	Subpainéis por categoria: `sns.FacetGrid(col, row, hue)`
Outliers	Método IQR: Q1 - 1.5×IQR a Q3 + 1.5×IQR
EDA	Checklist de 8 passos: dimensões → tipos → nulos → descritivas → distribuições → relações → segmentações → outliers

---

Glossário

Termo	Definição
API	Application Programming Interface — interface para comunicação entre sistemas
Array	Estrutura de dados multidimensional homogênea (NumPy)
Axis	Eixo de operação: 0 = linhas (colunas resultantes), 1 = colunas (linhas resultantes)
Broadcasting	Mecanismo NumPy para operar arrays de dimensões diferentes
Casting	Conversão de um tipo de dado para outro
Comprehension	Sintaxe concisa para criar listas, dicts ou sets
DataFrame	Tabela bidimensional do Pandas (linhas e colunas)
Decorator	Função que modifica o comportamento de outra função
EDA	Exploratory Data Analysis — análise exploratória de dados

Termo	Definição
Feature Engineering	Criar novas variáveis a partir das existentes
Generator	Função que produz valores sob demanda com `yield`
GroupBy	Operação de agrupar dados por categoria e aplicar função (Split-Apply-Combine)
IQR	Interquartile Range — Q3 minus Q1, usado para detectar outliers
KDE	Kernel Density Estimation — estimativa suavizada de distribuição
Lambda	Função anônima de uma linha
Merge	Combinar DataFrames por colunas comuns (equivalente a JOIN do SQL)
Outlier	Valor significativamente diferente do padrão dos dados
Pivot Table	Tabela de resumo que reorganiza dados (equivalente à tabela dinâmica do Excel)
Series	Array unidimensional rotulado do Pandas
Slicing	Fatiamento — selecionar subconjunto de elementos por posição
Vetorização	Aplicar operações a arrays inteiros sem loops explícitos
Window Function	Função que opera sobre janela de observações (rolling, expanding, shift)

Referências

Documentação Oficial

- **Python:** docs.python.org
- **NumPy:** numpy.org/doc
- **Pandas:** pandas.pydata.org/docs
- **Matplotlib:** matplotlib.org/stable
- **Seaborn:** seaborn.pydata.org

Livros e Materiais Complementares

- **Python Data Science Handbook** — Jake VanderPlas (jakevdp.github.io)
- **Python for Data Analysis** — Wes McKinney (criador do Pandas)
- **Storytelling with Data** — Cole Nussbaumer Knaflic (visualização)

Plataformas de Prática

- **Kaggle:** kaggle.com — datasets, competições e notebooks
- **Google Colab:** colab.research.google.com — ambiente gratuito com GPU
- **HackerRank:** hackerrank.com/domains/python — desafios de Python

---

Material elaborado para a disciplina Programação para Ciência de Dados — MBA Ciência de Dados, Universidade de Fortaleza (UNIFOR), 2026.